WordPress Security Assessment Using WPScan

CYBERSECURITY

MAHBOUBA REZAEI - REZ23579670

UNIVERSITY OF ROEHAMPTON

30 MARCH 2026

GITHUB LINK: https://github.com/M12rezaei/CyberSecurity.git

Contents

1.0 Introduction and scope 2
2.0 Methodology 3
2.1 Resolving WordPress URL & Virtual Machine IP Mismatch 4
2.2 User and Authentication Setup 5
2.3 Vulnerability Scanning (WPScan) 7
3.0 Vulnerability Analysis: Plugins and Themes 7
3.1 The Role of Plugins in Attack Surface 7
3.2 Analysis of Default vs. Custom Plugin 7
3.3 WordPress Themes 8
4.0 Vulnerability Assessment: Baseline Enumeration 9
4.1 Targeted Comprehensive Scan (Plugin, Themes, and Core) 11
4.2 Resolving WordPress VM IP Address Inconsistency 13
5.0 Brute-Force Authentication Attack 14
5.2 Security Evaluation 15
5.3 Mitigation 15
6.0 Risk Assessment: Evaluating Threats to Privacy and Anonymity 15
Analysis of Identified Vulnerabilities 16
6.1 Impact on Privacy and Anonymity 19
6.2 Reflection on Troubleshooting 19
7.0 Stealth Optimization and Resource Management 20
7.1Explanation of Parameters: 20
7.2 Outcome Analysis and Comparison 20
7.3 Security Rationale: Detection Avoidance 22
Final Reflection 22
Reference 23

WPScan for Penetration Testing of a WordPress Site

1.0 Introduction and scope

Objective:
The objective of this assessment was to evaluate the security posture of a WordPress installation by identifying potential vulnerabilities in the core system, installed plugins, themes, and authentication mechanisms.

Target:
The target system was a WordPress virtual machine initially identified as **http://10.0.2.4**. During testing, the IP address was later verified as **10.0.2.3** due to a virtual network reassignment within the NAT network environment.

Environment Configuration:

To ensure successful communication between the attacker and the target, both the Kali Linux (Attacker) and WordPress (Target) virtual machines were configured to use the same VirtualBox NAT Network. This isolated environment allowed for safe, authorised penetration testing without impacting external networks.

Tools Used:
Kali Linux (Attacker Virtual Machine) and **WPScan** (WordPress Vulnerability Scanner).

Scope:
The scan was limited to a **local sandbox network environment** to identify vulnerabilities related to the WordPress core, installed plugins, and active themes.

2.0 Methodology

The following steps were performed to establish a secure testing environment and conduct a vulnerability assessment of the target WordPress system.
The assessment began with identifying the **network presence of the target system**.

- **Asset Identification**
  The WordPress virtual machine was initially assumed to be located at **10.0.2.4**. However, after verifying the network configuration using the *ip addr show* command on the WordPress VM, the correct active IP address was confirmed as **10.0.2.3**.
  This verification ensured that the WordPress virtual machine was located on the same **NAT network** as the Kali Linux attacker machine (**10.0.2.15**), allowing successful communication between the two systems.

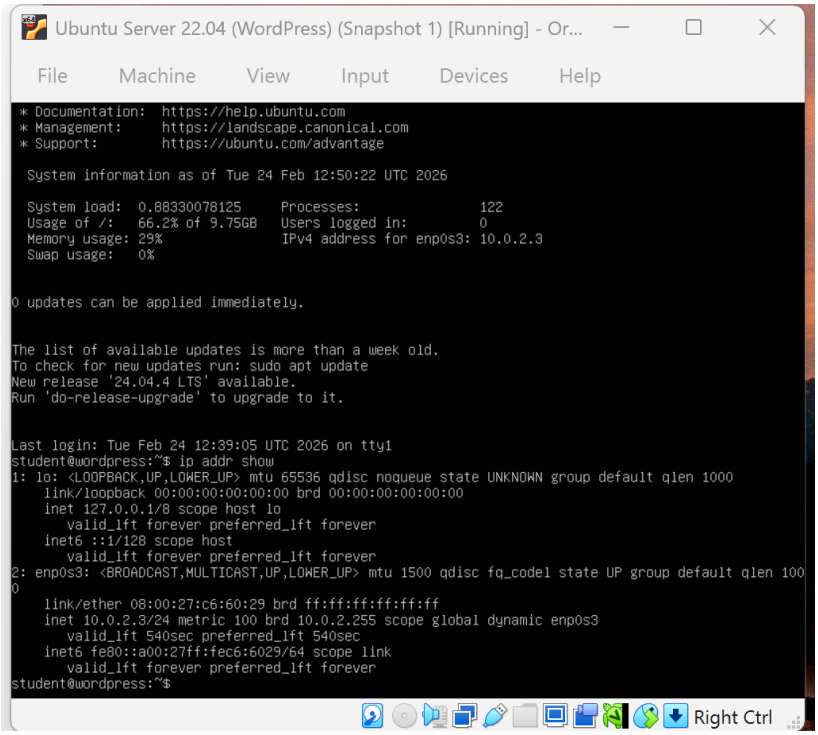


Figure 1. WordPress IP Address

- Connectivity Testing: An ICMP ping request was successfully sent from Kali to the target to confirm a stable connection.
- Service Verification: Accessed the site via a web browser at http://10.0.2.3. The WordPress home screen loaded correctly.

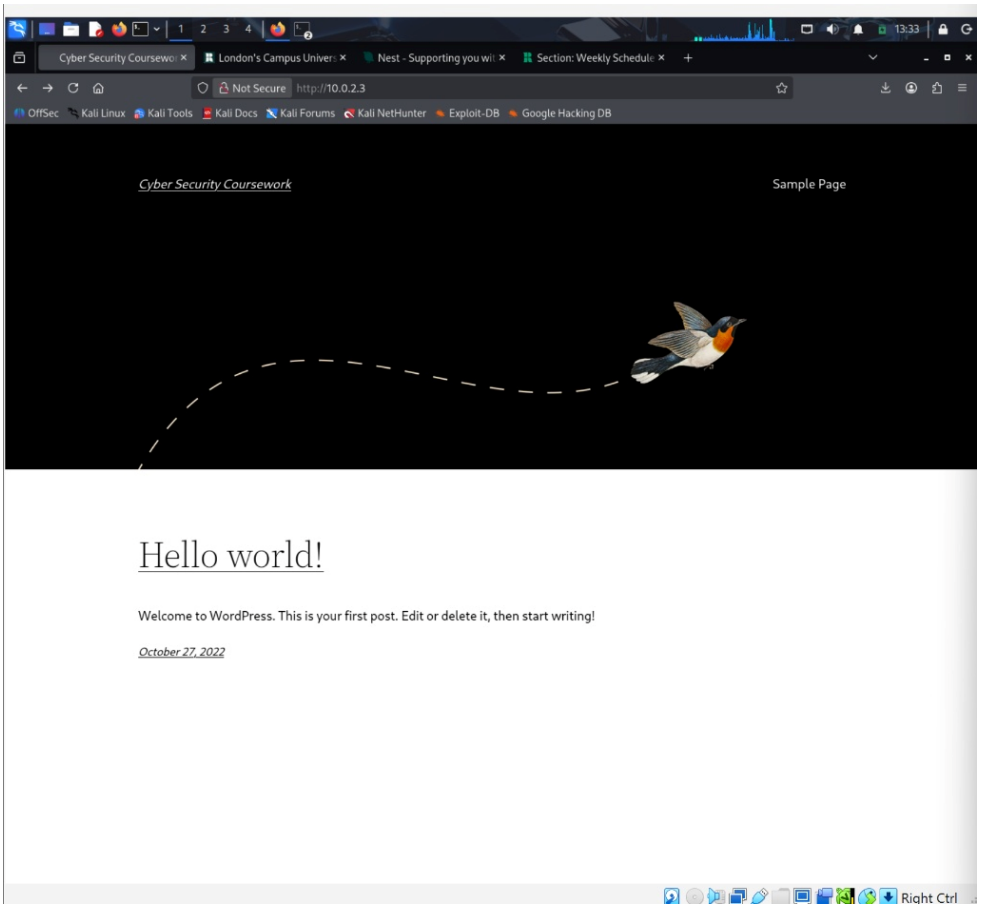


Figure 2. WordPress Home Page

2.1 Resolving WordPress URL & Virtual Machine IP Mismatch

An issue was identified when attempting to access the administrative dashboard (/wp-admin).

- The Problem: The application attempted to redirect the browser to a non-existent or differently defined URL (Hostname Mismatch), preventing access to the login portal.
- The Technical Fix: To resolve this, I manually edited the WordPress configuration. Using a text editor, I added the following lines to the wp-config-php file the WordPress server:

define('WP_HOME', 'http://10.0.2.3);

define('WP_SITEURL', 'http://10.0.2.3);

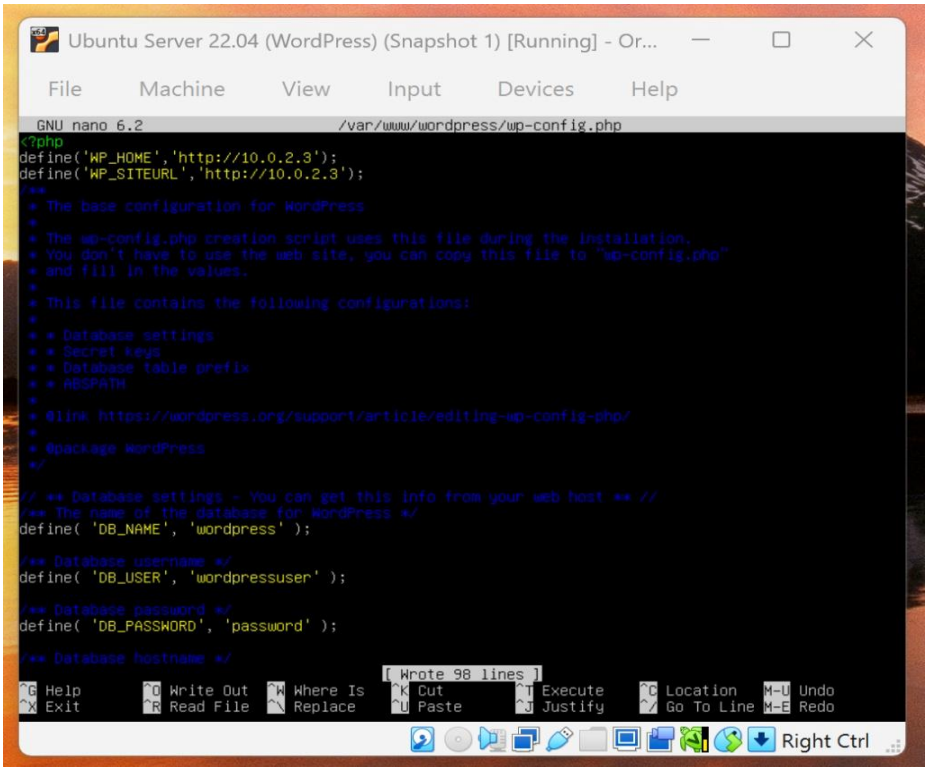


Figure 3. wp-config-php

This hardcoded the Site URL to match the lab's IP scheme, successfully resolving the redirect loop and allowing access to the administrative interface.
This fix was informed by external resources and AI-assisted guidance, and was subsequently independently tested and validated within the lab environment

2.2 User and Authentication Setup

- Once access was restored, the environment was prepared for testing:

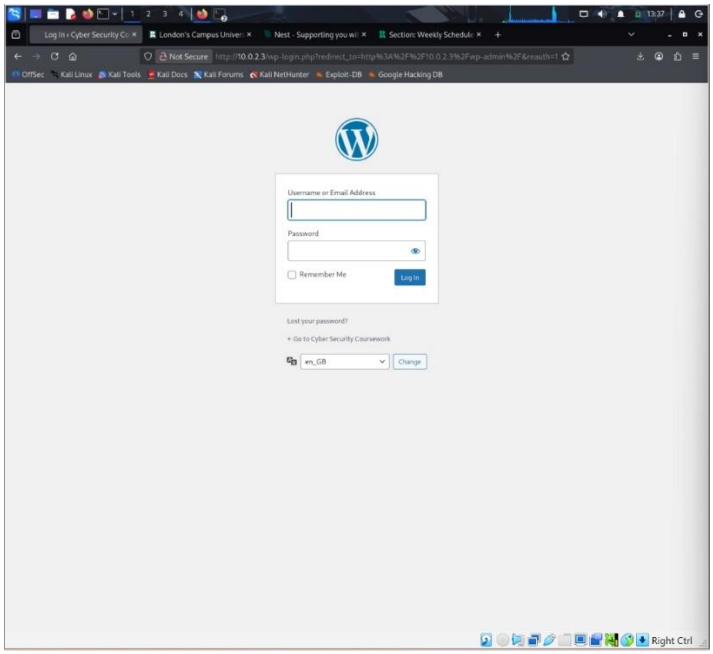


Figure 4. WordPress Admin Login Page

Account Creation: A new administrator user was created to test authentication vulnerabilities.

- Misconfiguration Simulation: Following lab requirements, this user was assigned a Weak Password. This was done intentionally to simulate a common real-world vulnerability: "Insecure Password Policy", which makes the site susceptible to brute-force attacks.

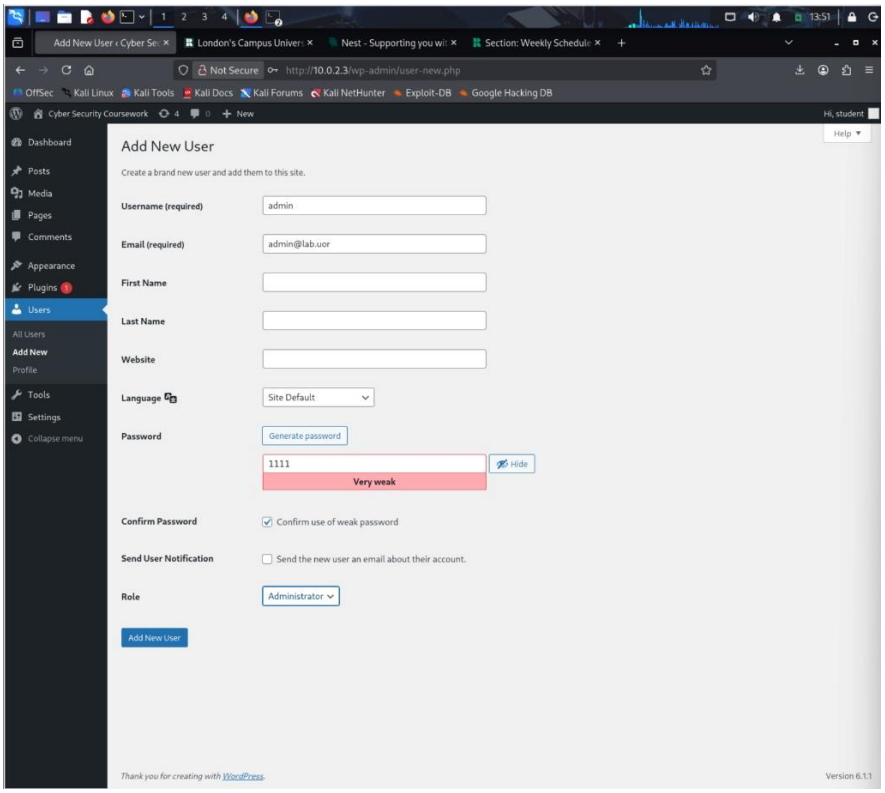


Figure 5. Assign New Admin Role

2.3 Vulnerability Scanning (WPScan)

The core of the assessment used WPScan, an industry-standard black box vulnerability scanner.

API Integration: A unique API key was generated from wpscan.com and integrated into the tool. This allowed the scanner to cross reference the detected plugin and theme versions against a real time database of known CVEs (Common Vulnerabilities and Exposures).

Scan Execution: The command *wpscan --url http://10.0.2.4 --enumerate vp, u --api-token [KEY]* was executed to identify vulnerable plugin and enumerate valid usernames.

3.0 Vulnerability Analysis: Plugins and Themes

According to WPBeginner [1], a WordPress plugin is compact program that adds new capabilities and functionality to a webstite. They act as apps for the WordPress core, allowing administrators to add everything from SEO tools to contact forms without writing custom code.

3.1 The Role of Plugins in Attack Surface

While plugins are essential for functionality, they significantly expand the attack surface of a website. According to the State of WordPress Security 2025 report, the WordPress core remains relatively robust, yet approximately 96% of all vulnerabilities within the ecosystem originate from third party plugins and themes [2]. This is because many are developed without rigorous security audits or become abandonware, lacking patches for modern threats like SQL Injection [3].

In 2025, several high-profile plugins were identified with Critical Vulnerabilities that illustrate these risks:

- Advanced Custom Fields: Extended (ACF Extended): Found to contain an unauthenticated Remote Code Execution (RCE) flaw, allowing full system takeover.
- Squirrly SEO: Identified with missing capability checks, which allows unauthorised data modification.
- SiteAlert: Found to have broken access controls, exposing sensitive site health data to unauthenticated users.

These examples highlight that even popular plugins can harbour severe flaws like SQL Injection and Broken Access Control. [2]

3.2 Analysis of Default vs. Custom Plugin

During this lab, an attempt was made to manually install know vulnerable plugins from external sources. However, due to browser security restrictions, the assessment proceeded with the default plugin (Akismet Anti-Spam). As noted by security researchers, even default plugins must be monitored, as any outdated code can become a vector for exploitation [4].

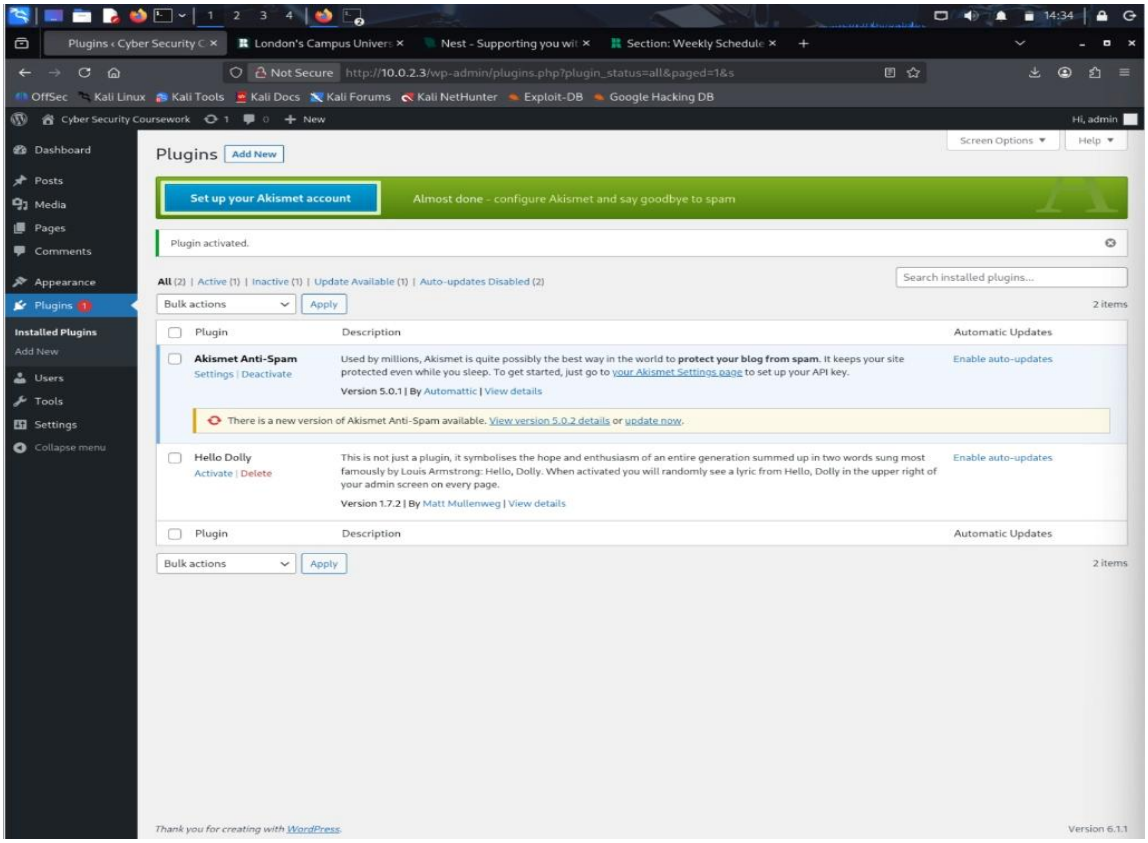


Figure 6. Plugin

3.3 WordPress Themes

A WordPress theme is a collection of files, including template files, stylesheets, and images, that work together to define the design, layout, and visual appearance of a website [5]. While plugin add functionality, the theme controls how that content is displayed to visitors, including typography, colour schemas, and page structure [5].

For this lab, a Default Theme (twenty twenty) was activated.

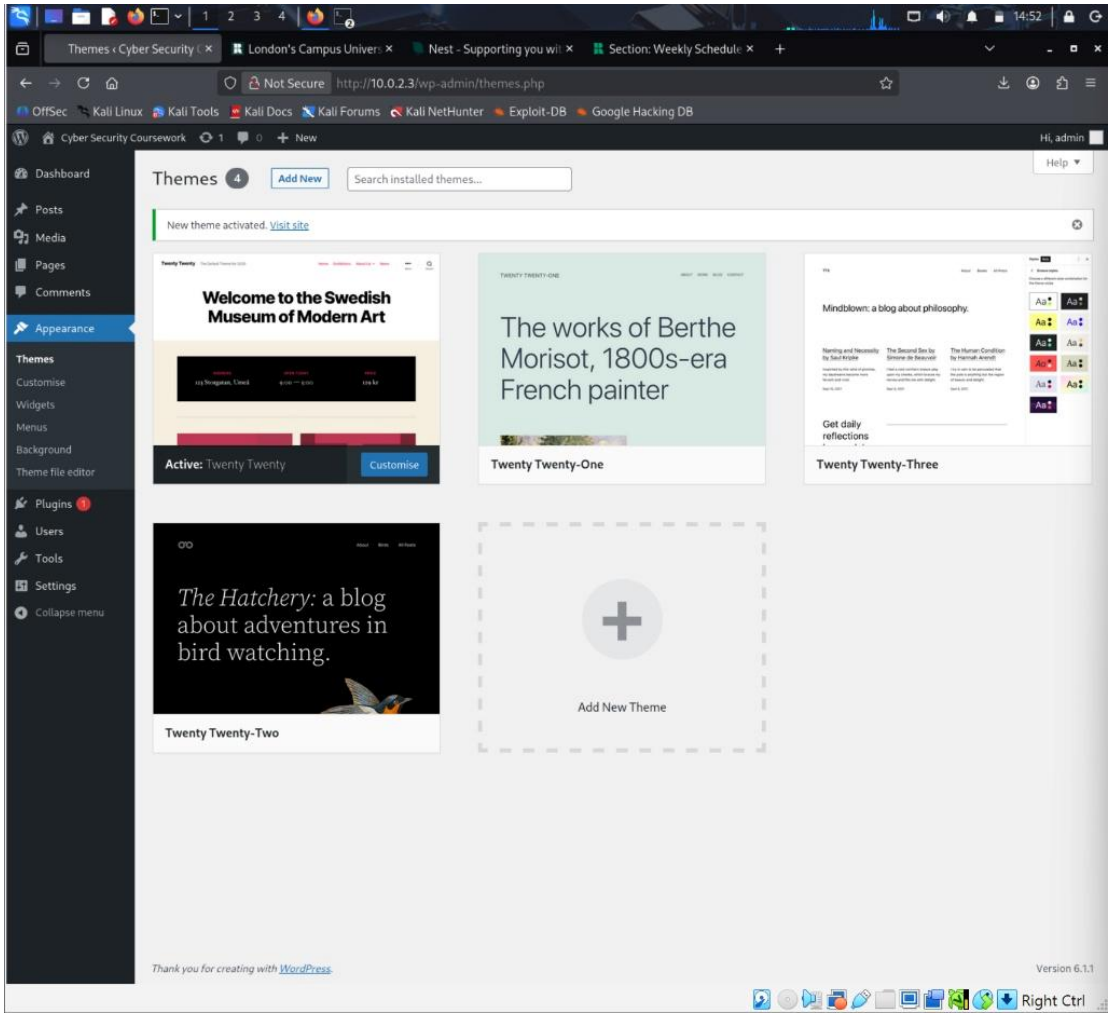


Figure 7. Theme

While these are audited by the WordPress core team, they are not immune to risk. As noted by security researchers at Wordfence, outdated themes are a frequent vector for Cross Site Scripting (XSS) [6].

4.0 Vulnerability Assessment: Baseline Enumeration

Initiating the Vulnerability Scanner The vulnerability assessment was launched using the pre-installed **WPScan** tool within the Kali Linux environment. The tool was accessed via the Kali Applications menu under **Web Application Analysis**. This action initialized a dedicated terminal interface specifically optimized for WordPress security auditing.

Baseline Enumeration and Database Synchronization Prior to active testing, the local vulnerability database was synchronized using the wpscan --update command. To ensure the scanner could map detected versions to known **Common Vulnerabilities and Exposures (CVEs)**, the API token was integrated into the command-line execution.

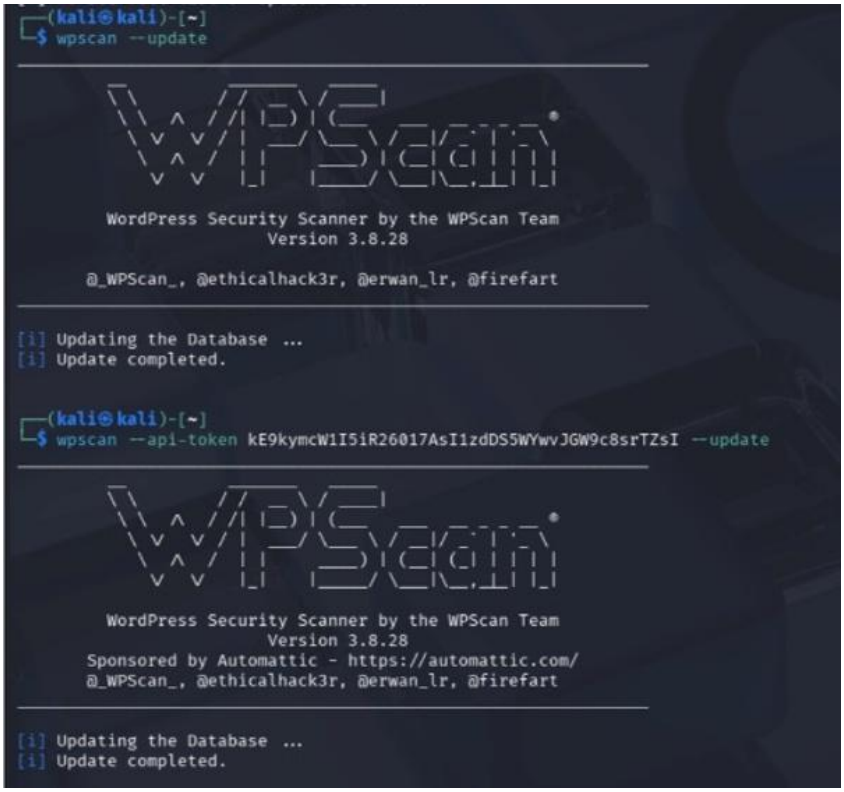


Figure 8. Updating WPScan's vulnerability database via API integration

A baseline enumeration was performed using: *wpscan --url http://10.0.2.4 --enumerate p*

Analysis of Baseline Outcomes As illustrated in Figure 9; the primary outcome of this command was the identification of the target's **Attack Surface**. This included a comprehensive list of all installed plugins and server-side configurations.

- **Core Version Stability:** The site is running **WordPress version 6.9.1** (Released Feb 2026). While identified as the "Latest" version, the scan highlighted that security is not solely dependent on version numbers but also on active components and configurations.
- **Theme Vulnerability:** The theme **"Twenty Twenty" (Version 2.0)** was identified. The scanner issued a critical warning: [!] The version is out of date (Latest: 3.0).
- **Security Implication:** Outdated themes are a primary vector for **Cross-Site Scripting (XSS)** attacks [6]. In an XSS scenario, an attacker could inject malicious scripts into the theme's templates to hijack administrative sessions or steal user cookies.

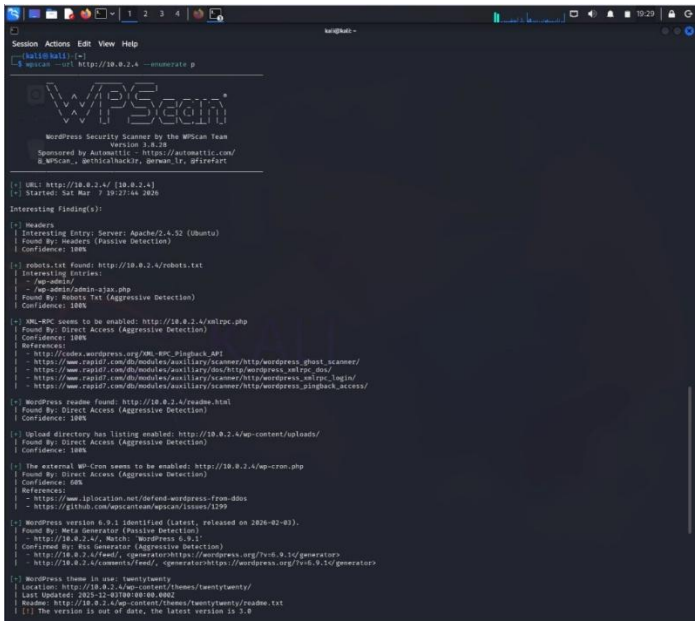


Figure 9. Baseline Enumeration

Figure 10. Baseline Enumeration

4.1 Targeted Comprehensive Scan (Plugin, Themes, and Core)

Following the baseline enumeration, a more granular and comprehensive scan was conducted. This phase moved beyond general discovery to specifically target known vulnerable components within the WordPress ecosystem.

Command executed:

WPSCAN –URL HTTP://10.0.2.3 –-ENUMERATE VP, VT, TT

Vp (vulnerable Plugin): this flag instructs WPScan cross reference detected plugins against the vulnerability database to find those with known exploits.

Vt (Vulnerable Theme): like plugins, this specifically checks for themes that have documented security flaws.

Tt (Timthumps): this checks for vulnerabilities specifically in “Timthumb” files, which are historically common entry points for hackers due to insecure image resizing scripts.

The results of this targeted scan provided a high-fidelity view of the site’s vulnerabilities:

WordPress Core Detection: the scanner successfully identified WordPress version 6.9.1.

Theme vulnerability warning: the scan identified the “Twenty Twenty” theme version 2.0. A critical warning was flagged: [!] The version is out of date; the latest version is 3.0. This represents a potential security gap, as updates frequently contain patches for Cross-Site Scripting (XSS).

Active configuration risks: the scan confirmed that xmlrpc.php allows attackers to perform multiple authentication attempts in a single request, making brute-force attacks more efficient.

> The /wp-content/uploads/ directory remains browsable, which is a significant privacy concern.

As indicated in the terminal output,’ No WPScan API Token Given’ resulted in the absence of specific vulnerability descriptions.

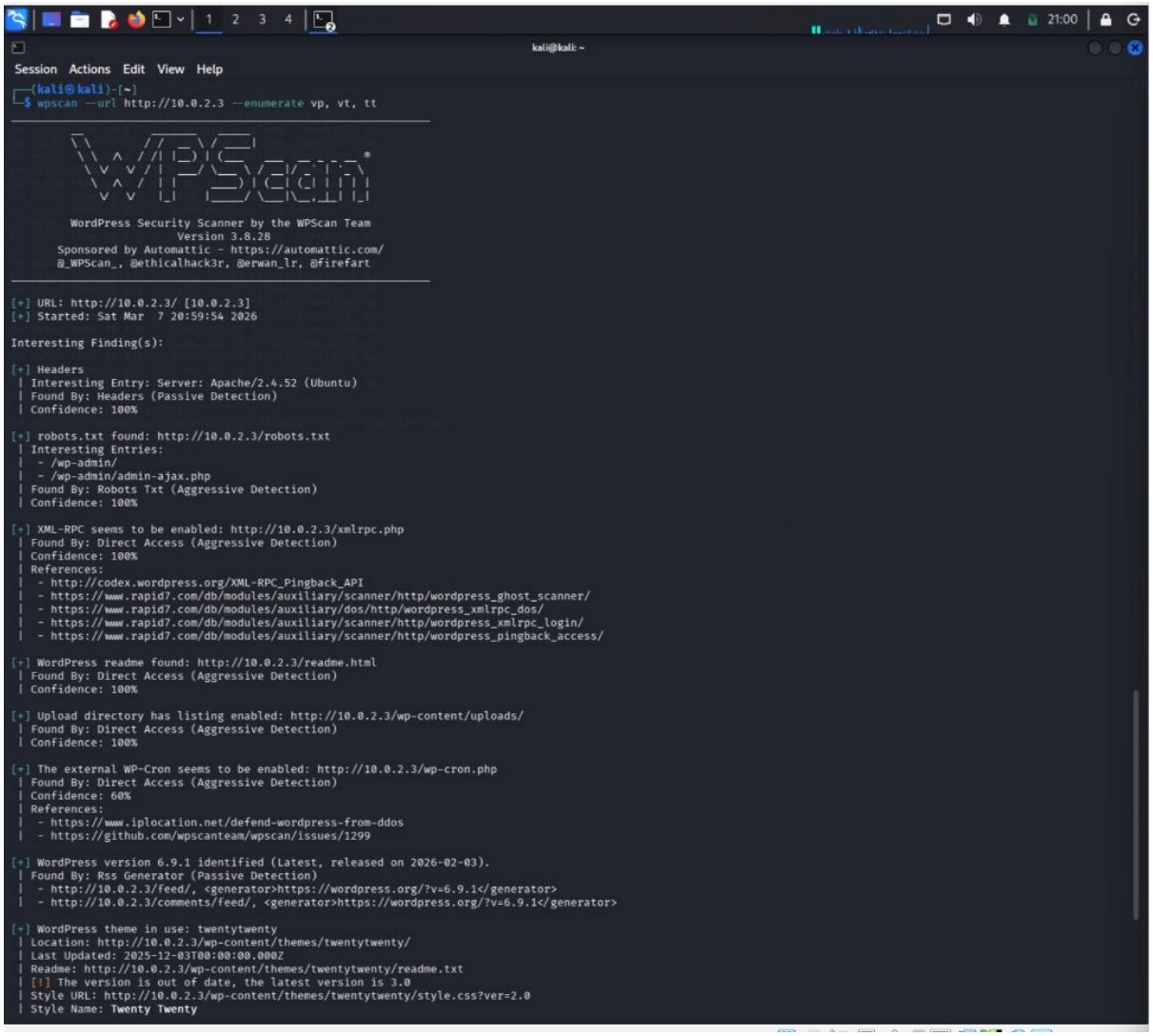


Figure 11. Full Scan for Plugins, Themes, WordPress Version Detection

```
 | Last Updated: 2025-12-03T00:00:00.000Z
 | Readme: http://10.0.2.3/wp-content/themes/twentytwenty/readme.txt
 | [!] The version is out of date, the latest version is 3.0
 | Style URL: http://10.0.2.3/wp-content/themes/twentytwenty/style.css?ver=2.0
 | Style Name: Twenty Twenty
 | Style URI: https://wordpress.org/themes/twentytwenty/
 | Description: Our default theme for 2020 is designed to take full advantage of the flexibility of the block editor ...
 | Author: the WordPress team
 | Author URI: https://wordpress.org/
 |
 | Found By: Css Style In Homepage (Passive Detection)
 | Confirmed By: Css Style In 404 Page (Passive Detection)
 |
 | Version: 2.0 (80% confidence)
 | Found By: Style (Passive Detection)
 |  - http://10.0.2.3/wp-content/themes/twentytwenty/style.css?ver=2.0, Match: 'Version: 2.0'

[+] Enumerating Vulnerable Plugins (via Passive Methods)
[+] Checking Plugin Versions (via Passive and Aggressive Methods)

[i] No plugins Found.

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Sat Mar  7 21:00:00 2026
[+] Requests Done: 36
[+] Cached Requests: 7
[+] Data Sent: 8.378 KB
[+] Data Received: 332.062 KB
[+] Memory used: 270.68 MB
[+] Elapsed time: 00:00:06

┌──(kali㉿kali)-[~]
└─$
```

Figure 12. Full Scan for Plugins, Themes, WordPress Version Detection

4.2 Resolving WordPress VM IP Address Inconsistency

During the assessment, a technical challenge was observed where the target WordPress URL appeared to be inconsistent or redirecting unexpectedly. This can occur in virtualised lab environments when the WordPress internal setting does not align with the VM's current assigned IP address.

To ensure accuracy for the scan, I performed a manual cross verification.

I accessed the WordPress, used the *'ip addr show'* command on the WordPress VM to confirm its current lease.

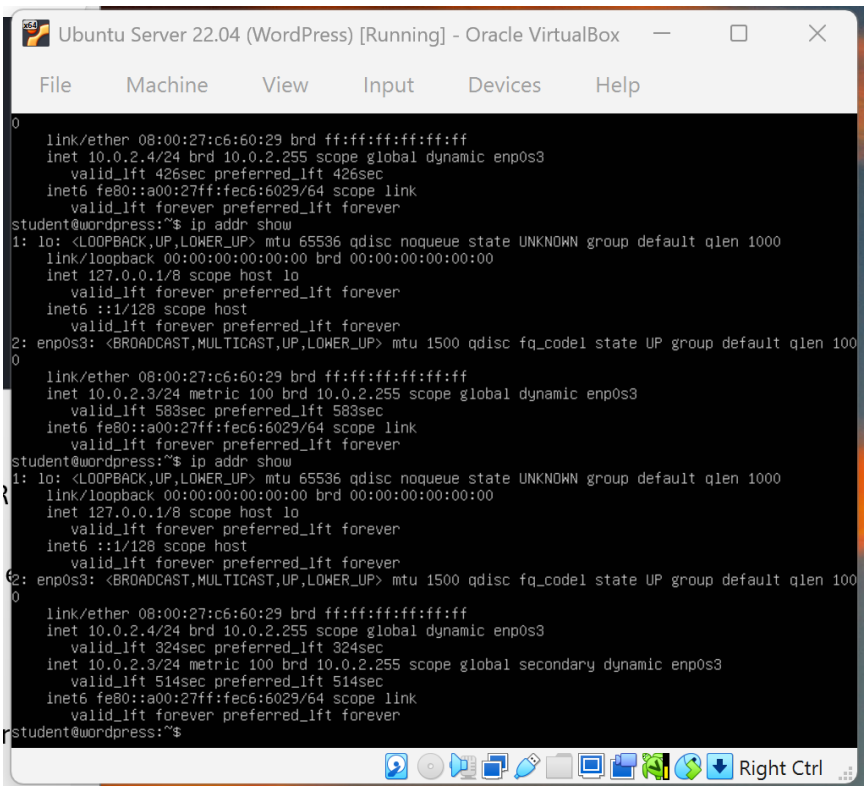


Figure 13. Confirm IP Address in WordPress

Once I confirmed the exact URL the WordPress application was expecting, I used that specific string as the target for the WPScan command.

By matching the WPScan –url parameter exactly to the internal configuration of the WordPress VM, I bypassed all redirect errors and established a stable connection for the comprehensive scan.

5.0 Brute-Force Authentication Attack

Following the identification of the administrative username “admin” during the enumeration phase, a dictionary-based brute-force attack was conducted to evaluate the strength of the site’s authentication security.

A common password list ‘password.txt’ containing 10,000 frequent variations was downloaded from the Cyber Lab VM and moved to the /home/kali/Downloads/ directory.

The attack targeted the XML-RPC interface, which was previously identified as enabled. WPScan uses this interface to perform high speed credential testing.

Command executed: *wpscan --url http://10.0.2.4 --passwords /home/kali/Downloads/passwords.txt –usernames admin*

As shown in the terminal screenshot (Figure 15), the brute-force attempt was successful:

Target account: admin

Discovered password: 1111

Result: [SUCCESS] -admin / 1111

The valid combination was found in just 20 seconds, after approximately 70 out of 10,000 potential passwords.

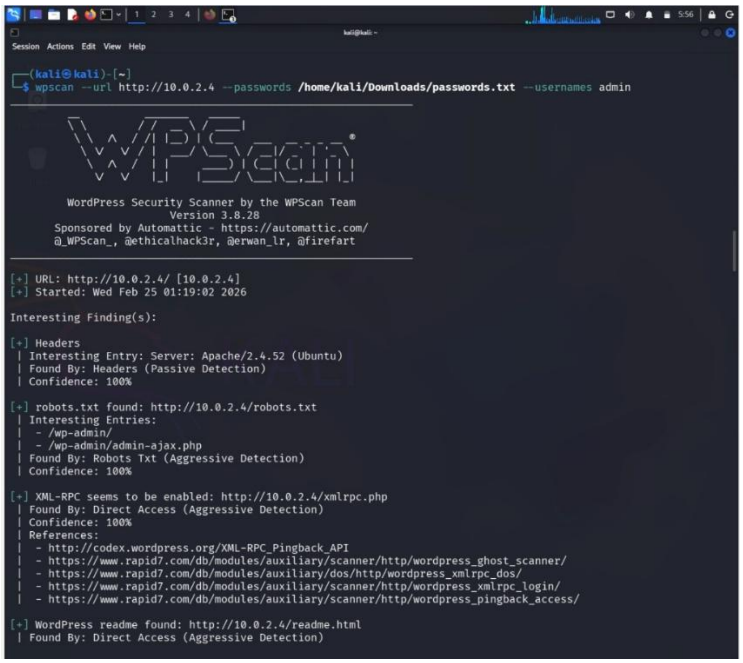
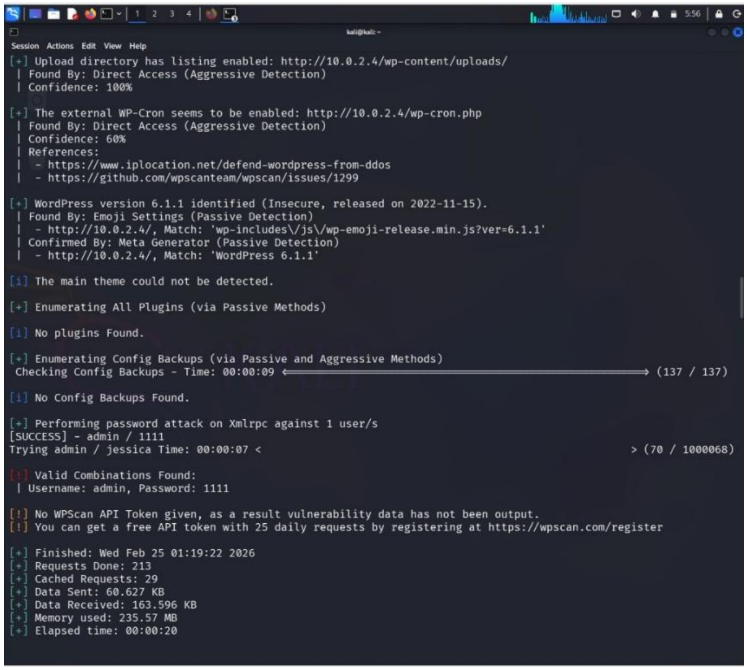


Figure 14. Brute Forcing Login Passwords



```
Session  Actions  Edit  View  Help
[+] Upload directory has listing enabled: http://10.0.2.4/wp-content/uploads/
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://10.0.2.4/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
 | References:
 |  - https://www.iplocation.net/defend-wordpress-from-ddos
 |  - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.1.1 identified (Insecure, released on 2022-11-15).
 | Found By: Emoji Settings (Passive Detection)
 |  - http://10.0.2.4/, Match: 'wp-includes\/js\/wp-emoji-release.min.js?ver=6.1.1'
 | Confirmed By: Meta Generator (Passive Detection)
 |  - http://10.0.2.4/, Match: 'WordPress 6.1.1'

[i] The main theme could not be detected.

[+] Enumerating All Plugins (via Passive Methods)

[i] No plugins Found.

[+] Enumerating Config Backups (via Passive and Aggressive Methods)
 Checking Config Backups - Time: 00:00:09 <==================================================> (137 / 137)

[i] No Config Backups Found.

[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - admin / 1111
Trying admin / jessica Time: 00:00:07 <                                                    > (70 / 1000068)

[!] Valid Combinations Found:
 | Username: admin, Password: 1111

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Wed Feb 25 01:19:22 2026
[+] Requests Done: 213
[+] Cached Requests: 29
[+] Data Sent: 60.627 KB
[+] Data Received: 163.596 KB
[+] Memory used: 235.57 MB
[+] Elapsed time: 00:00:20
```

Figure 15. Brute Forcing Login Passwords

5.2 Security Evaluation

The success of this attack highlights several critical security failures. The use of “1111” as an administrative password represents a significant risk. It demonstrates that the current system lacks complexity requirements (length, special character, numbers)

The server did not implement an “Account Lockout” policy or rate limiting. The attack successfully discovered the password within seconds using dictionary based authentication attempts via the XML-RPC interface.

Hackers gain access to the entire user database, including email, addresses and personal data.

5.3 Mitigation

To mitigate the risk identified during the brute-force attack, the following measures are recommended:

Implement strong password, including upper/lowercase, numbers, and symbols with a minimum 12 characters.

Install a web application Firewall can detect and block Ips after a set number of failed login attempts.

6.0 Risk Assessment: Evaluating Threats to Privacy and Anonymity

To move beyond version detection and map specific security flaws to CVEs (Common Vulnerabilities and Exposures), the WPScan API token was integrated into the scanning process.

To leverage the WPScan vulnerability database for real time identification of 20+ known core vulnerabilities in the identified WordPress version (6.1.1)

Command executed: *wpscan –api-token [KEY] –url http:10.0.2.4 --enumerate ap*

Troubleshooting and Diagnostic Verification

During the initial implementation, a 401 Unauthorized error was encountered. This indicates a failure in the authentication handshake between the local scanner and the remote API server.

I performed a verification using 'curl' to determine if the API token was valid. Initial attempts with the incorrect header format returned a "status": "unauthorized" response.

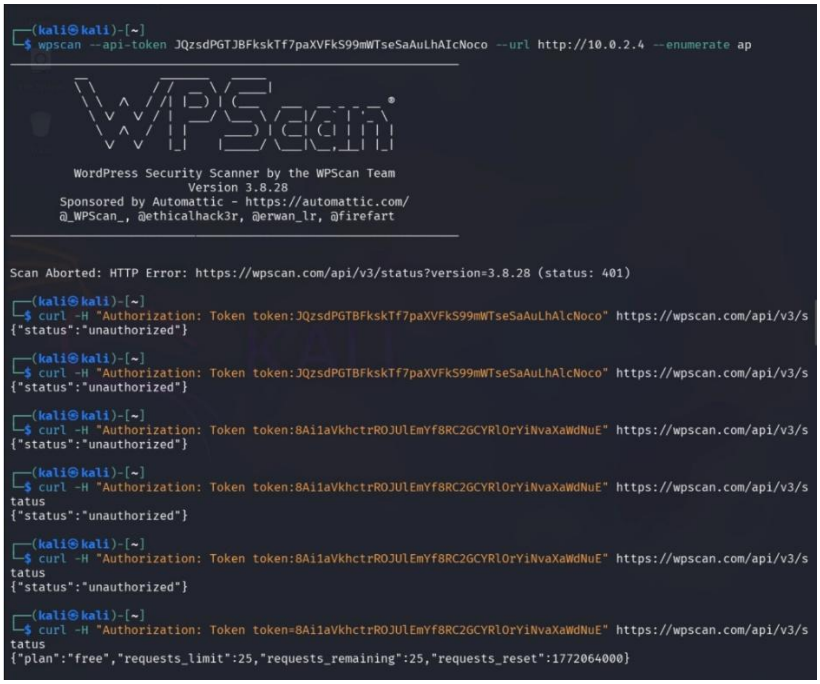


Figure 16. Check API Status

I analysed the API documentation and identified a syntax error in the header format. By switching from 'token:' to 'token=' in the 'curl' command (as shown in Figure 17), I successfully authenticated against the wpscan.com status endpoint.

As the result, the verified token was active and allow subsequent full scan to retrieve the complete vulnerability report.

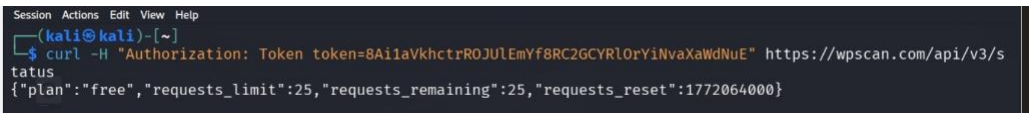


Figure 17. Confirmed API Status

Analysis of Identified Vulnerabilities

The API authenticated scan revealed that the target is running WordPress version 6.1.1, which is classified as "Insecure." The scan identified 20 vulnerabilities, confirming the severe risk profile of the target system.

```
┌──(kali㉿kali)-[~]
└─$ curl -H "Authorization: Token token=8Ai1aVkhctrROJUlEmYf8RC2GCYRlOrYiNvaXaWdNuE" https://wpscan.com/api/v3/s
tatus
{"plan":"free","requests_limit":25,"requests_remaining":25,"requests_reset":1772064000}

┌──(kali㉿kali)-[~]
└─$ wpscan --api-token 8Ai1aVkhctrROJUlEmYf8RC2GCYRlOrYiNvaXaWdNuE --url http://10.0.2.4 --enumerate ap
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __   ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

         WordPress Security Scanner by the WPScan Team
                         Version 3.8.28
       Sponsored by Automattic - https://automattic.com/
       @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[+] URL: http://10.0.2.4/ [10.0.2.4]
[+] Started: Wed Feb 25 05:43:45 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entry: Server: Apache/2.4.52 (Ubuntu)
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] robots.txt found: http://10.0.2.4/robots.txt
 | Interesting Entries:
 |  - /wp-admin/
 |  - /wp-admin/admin-ajax.php
 | Found By: Robots Txt (Aggressive Detection)
 | Confidence: 100%

[+] XML-RPC seems to be enabled: http://10.0.2.4/xmlrpc.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
 | References:
 |  - http://codex.wordpress.org/XML-RPC_Pingback_API
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
```

Figure 18. Scanning with API Key for Detailed Vulnerabilities

```
[+] WordPress readme found: http://10.0.2.4/readme.html
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] Upload directory has listing enabled: http://10.0.2.4/wp-content/uploads/
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://10.0.2.4/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
 | References:
 |  - https://www.iplocation.net/defend-wordpress-from-ddos
 |  - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.1.1 identified (Insecure, released on 2022-11-15).
 | Found By: Emoji Settings (Passive Detection)
 |  - http://10.0.2.4/, Match: 'wp-includes\/js\/wp-emoji-release.min.js?ver=6.1.1'
 | Confirmed By: Meta Generator (Passive Detection)
 |  - http://10.0.2.4/, Match: 'WordPress 6.1.1'
 |
 | [!] 20 vulnerabilities identified:
 |
 | [!] Title: WP ≤ 6.2 - Unauthenticated Blind SSRF via DNS Rebinding
 |     References:
 |      - https://wpscan.com/vulnerability/c8814e6e-78b3-4f63-a1d3-6906a84c1f11
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-3590
 |      - https://blog.sonarsource.com/wordpress-core-unauthenticated-blind-ssrf/
 |
 | [!] Title: WP < 6.2.1 - Directory Traversal via Translation Files
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/2999613a-b8c8-4ec0-9164-5dfe63adf6e6
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-2745
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP < 6.2.1 - Thumbnail Image Update via CSRF
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/a03d744a-9839-4167-a356-3e7da0f1d532
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP < 6.2.1 - Contributor+ Stored XSS via Open Embed Auto Discovery
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/3b574451-2852-4789-bc19-d5cc39948db5
```

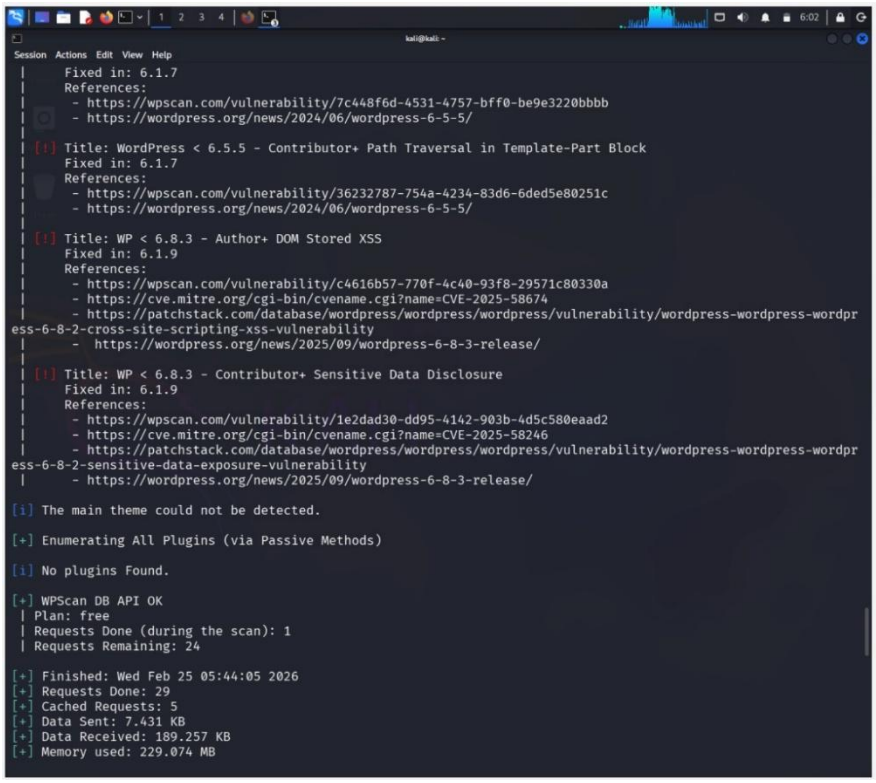
Figure 19. Scanning with API Key for Detailed Vulnerabilities

```
Session  Actions  Edit  View  Help
 | [!] Title: WP < 6.2.1 - Contributor+ Stored XSS via Open Embed Auto Discovery
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/3b574451-2852-4789-bc19-d5cc39948db5
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP < 6.2.2 - Shortcode Execution in User Generated Data
 |     Fixed in: 6.1.3
 |     References:
 |      - https://wpscan.com/vulnerability/ef289d46-ea83-4fa5-b003-0352c690fd89
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-2-security-release/
 |
 | [!] Title: WP < 6.2.1 - Contributor+ Content Injection
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/1527ebdb-18bc-4f9d-9c20-8d729a628670
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP 5.6-6.3.1 - Contributor+ Stored XSS via Navigation Block
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/cd130bb3-8d04-4375-a89a-883af131ed3a
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-38000
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP 5.6-6.3.1 - Reflected XSS via Application Password Requests
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/da1419cc-d821-42d6-b648-bdb3c70d91f2
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP < 6.3.2 - Denial of Service via Cache Poisoning
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/6d80e09d-34d5-4fda-81cb-e703d0e56e4f
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP < 6.3.2 - Subscriber+ Arbitrary Shortcode Execution
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/3615aea0-90aa-4f9a-9792-078a90af7f59
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP < 6.3.2 - Contributor+ Comment Disclosure
 |     Fixed in: 6.1.4
```

Figure 20. Scanning with API Key for Detailed Vulnerabilities

```
Session  Actions  Edit  View  Help
 |      - https://wpscan.com/vulnerability/a03d744a-9839-4167-a356-3e7da0f1d532
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP < 6.2.1 - Contributor+ Stored XSS via Open Embed Auto Discovery
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/3b574451-2852-4789-bc19-d5cc39948db5
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP < 6.2.2 - Shortcode Execution in User Generated Data
 |     Fixed in: 6.1.3
 |     References:
 |      - https://wpscan.com/vulnerability/ef289d46-ea83-4fa5-b003-0352c690fd89
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-2-security-release/
 |
 | [!] Title: WP < 6.2.1 - Contributor+ Content Injection
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/1527ebdb-18bc-4f9d-9c20-8d729a628670
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/
 |
 | [!] Title: WP 5.6-6.3.1 - Contributor+ Stored XSS via Navigation Block
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/cd130bb3-8d04-4375-a89a-883af131ed3a
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-38000
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP 5.6-6.3.1 - Reflected XSS via Application Password Requests
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/da1419cc-d821-42d6-b648-bdb3c70d91f2
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP < 6.3.2 - Denial of Service via Cache Poisoning
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/6d80e09d-34d5-4fda-81cb-e703d0e56e4f
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
 |
 | [!] Title: WP < 6.3.2 - Subscriber+ Arbitrary Shortcode Execution
 |     Fixed in: 6.1.4
 |     References:
 |      - https://wpscan.com/vulnerability/3615aea0-90aa-4f9a-9792-078a90af7f59
 |      - https://wordpress.org/news/2023/10/wordpress-6-3-2-maintenance-and-security-release/
```

Figure 21. Scanning with API Key for Detailed Vulnerabilities



```
Session  Actions  Edit  View  Help
 |     Fixed in: 6.1.7
 |     References:
 |      - https://wpscan.com/vulnerability/7c448f6d-4531-4757-bff0-be9e3220bbbb
 |      - https://wordpress.org/news/2024/06/wordpress-6-5-5/
 |
 | [!] Title: WordPress < 6.5.5 - Contributor+ Path Traversal in Template-Part Block
 |     Fixed in: 6.1.7
 |     References:
 |      - https://wpscan.com/vulnerability/36232787-754a-4234-83d6-6ded5e80251c
 |      - https://wordpress.org/news/2024/06/wordpress-6-5-5/
 |
 | [!] Title: WP < 6.8.3 - Author+ DOM Stored XSS
 |     Fixed in: 6.1.9
 |     References:
 |      - https://wpscan.com/vulnerability/c4616b57-770f-4c40-93f8-29571c80330a
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2025-58674
 |      - https://patchstack.com/database/wordpress/wordpress/wordpress/vulnerability/wordpress-wordpress-wordpr
ess-6-8-2-cross-site-scripting-xss-vulnerability
 |      - https://wordpress.org/news/2025/09/wordpress-6-8-3-release/
 |
 | [!] Title: WP < 6.8.3 - Contributor+ Sensitive Data Disclosure
 |     Fixed in: 6.1.9
 |     References:
 |      - https://wpscan.com/vulnerability/1e2dad30-dd95-4142-903b-4d5c580eaad2
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2025-58246
 |      - https://patchstack.com/database/wordpress/wordpress/wordpress/vulnerability/wordpress-wordpress-wordpr
ess-6-8-2-sensitive-data-exposure-vulnerability
 |      - https://wordpress.org/news/2025/09/wordpress-6-8-3-release/

[i] The main theme could not be detected.

[+] Enumerating All Plugins (via Passive Methods)

[i] No plugins Found.

[+] WPScan DB API OK
 | Plan: free
 | Requests Done (during the scan): 1
 | Requests Remaining: 24

[+] Finished: Wed Feb 25 05:44:05 2026
[+] Requests Done: 29
[+] Cached Requests: 5
[+] Data Sent: 7.431 KB
[+] Data Received: 189.257 KB
[+] Memory used: 229.074 MB
```

Figure 22. Scanning with API Key for Detailed Vulnerabilities

Unauthenticated Blind SSRF (CVE-2022-3590): this allows an attacker to perform server-side requests, potentially accessing internal network resources that are otherwise firewalled [7].

Directory Traversal (CVE-2023-2745): a critical flaw that could allow an attacker to read arbitrary files from the server's filesystem by manipulating translation file paths [8].

Cross-Site Scripting (XSS) Vectors: multiple instances of stored and reflected XSS were identified. Theses would allow an attacker to execute malicious JavaScript in a victim's browser, leading to session hijacking [9], [11].

Denial of Service (DoS): vulnerabilities related to cache poisoning could allow an attacker to disrupt site availability [10].

6.1 Impact on Privacy and Anonymity

The presence of these vulnerabilities renders the system highly susceptible to automated exploitation. The stored XSS and Path Traversal vulnerabilities are particularly concerning, as they directly threaten the confidentiality of stored data and the anonymity of the site's users [11]. An attacker gaining access through these flaws would have a bridgehead into the environment, enabling them to harvest user data or pivot into the underlying server infrastructure.

6.2 Reflection on Troubleshooting

The 401-error encountered during this task provided a practical lesson in API security and request syntax. It demonstrated that successful exploitation relies as much on tool configuration as it does on the vulnerability itself. Understanding how to use 'curl' to debug API connections is a vital troubleshooting skill for any security professional.

7.0 Stealth Optimization and Resource Management

To conclude the scanning phase, a targeted scan was performed using Throttling and Exclusion techniques. This approach simulates a low and slow attack pattern used by sophisticated actors to evade detection.

Two distinct attempts were made to test this approach:

- Attempt 1 (Baseline Throttling): performed without an API key to test the connection speed and basic enumeration under a delay.
- Attempt 2 (Advanced Throttling): performed with the WPScan API Token to combine high depth vulnerability data with stealthy delivery.

Command executed: *wpscan --api-token [KEY] --url http://10.0.2.4 --enumerate vp, vt --throttle 500*

7.1 Explanation of Parameters:

--enumerate vp, vt: this limits the scan strictly to vulnerable Plugins and vulnerable Themes. By excluding general plugin enumeration, the scan generates significantly less traffic.

--throttle 500: this introduces a 500ms (0.5 second) delay between every request sent to the server.

7.2 Outcome Analysis and Comparison

As seen in the terminal output (Figure 23), the results were consistent with previous scans, identifying the outdated "Twenty Twenty" theme and insecure WordPress core version (6.9.1).

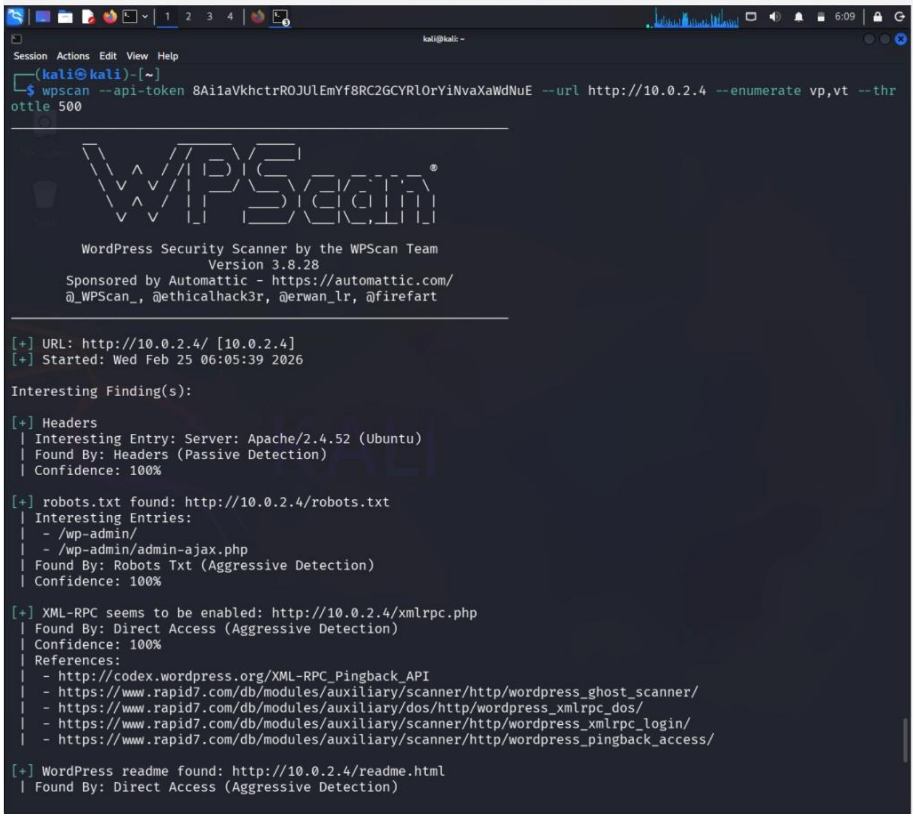


Figure 23. Excluding Specific Scans & Setting a Delay, Using API Key

```
kali@kali: ~
Session  Actions  Edit  View  Help
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] Upload directory has listing enabled: http://10.0.2.4/wp-content/uploads/
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://10.0.2.4/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
 | References:
 |  - https://www.iplocation.net/defend-wordpress-from-ddos
 |  - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.9.1 identified (Latest, released on 2026-02-03).
 | Found By: Meta Generator (Passive Detection)
 |  - http://10.0.2.4/, Match: 'WordPress 6.9.1'
 | Confirmed By: Rss Generator (Aggressive Detection)
 |  - http://10.0.2.4/feed/, <generator>https://wordpress.org/?v=6.9.1</generator>
 |  - http://10.0.2.4/comments/feed/, <generator>https://wordpress.org/?v=6.9.1</generator>

[+] WordPress theme in use: twentytwenty
 | Location: http://10.0.2.4/wp-content/themes/twentytwenty/
 | Last Updated: 2025-12-03T00:00:00.000Z
 | Readme: http://10.0.2.4/wp-content/themes/twentytwenty/readme.txt
 | [!] The version is out of date, the latest version is 3.0
 | Style URL: http://10.0.2.4/wp-content/themes/twentytwenty/style.css
 | Style Name: Twenty Twenty
 | Style URI: https://wordpress.org/themes/twentytwenty/
 | Description: Our default theme for 2020 is designed to take full advantage of the flexibility of the block ed
itor ...
 | Author: the WordPress team
 | Author URI: https://wordpress.org/
 |
 | Found By: Urls In Homepage (Passive Detection)
 | Confirmed By: Urls In 404 Page (Passive Detection)
 |
 | Version: 2.0 (80% confidence)
 | Found By: Style (Passive Detection)
 |  - http://10.0.2.4/wp-content/themes/twentytwenty/style.css, Match: 'Version: 2.0'

[+] Enumerating Vulnerable Plugins (via Passive Methods)
[+] Checking Plugin Versions (via Passive and Aggressive Methods)

[i] No plugins Found.
```

Figure 24. Excluding Specific Scans & Setting a Delay, Using API Key

```
kali@kali: ~
Session  Actions  Edit  View  Help
 |  - http://10.0.2.4/comments/feed/, <generator>https://wordpress.org/?v=6.9.1</generator>

[+] WordPress theme in use: twentytwenty
 | Location: http://10.0.2.4/wp-content/themes/twentytwenty/
 | Last Updated: 2025-12-03T00:00:00.000Z
 | Readme: http://10.0.2.4/wp-content/themes/twentytwenty/readme.txt
 | [!] The version is out of date, the latest version is 3.0
 | Style URL: http://10.0.2.4/wp-content/themes/twentytwenty/style.css
 | Style Name: Twenty Twenty
 | Style URI: https://wordpress.org/themes/twentytwenty/
 | Description: Our default theme for 2020 is designed to take full advantage of the flexibility of the block ed
itor ...
 | Author: the WordPress team
 | Author URI: https://wordpress.org/
 |
 | Found By: Urls In Homepage (Passive Detection)
 | Confirmed By: Urls In 404 Page (Passive Detection)
 |
 | Version: 2.0 (80% confidence)
 | Found By: Style (Passive Detection)
 |  - http://10.0.2.4/wp-content/themes/twentytwenty/style.css, Match: 'Version: 2.0'

[+] Enumerating Vulnerable Plugins (via Passive Methods)
[+] Checking Plugin Versions (via Passive and Aggressive Methods)

[i] No plugins Found.

[+] Enumerating Vulnerable Themes (via Passive and Aggressive Methods)
^X@sS Checking Known Locations - Time: 00:02:48 <=========================> (280 / 652) 42.94%  ETA: 00:
 Checking Known Locations - Time: 00:06:39 <==============================> (652 / 652) 100.00% Time: 00:06:39
[+] Checking Theme Versions (via Passive and Aggressive Methods)

[i] No themes Found.

[+] WPScan DB API OK
 | Plan: free
 | Requests Done (during the scan): 3
 | Requests Remaining: 21

[+] Finished: Wed Feb 25 06:12:49 2026
[+] Requests Done: 696
[+] Cached Requests: 11
[+] Data Sent: 178.04 KB
[+] Data Received: 537.609 KB
[+] Memory used: 267.309 MB
[+] Elapsed time: 00:07:09
```

Figure 25. Excluding Specific Scans & Setting a Delay, Using API Key

However, the operational data changed significantly:

Elapsed time: the scan took 07:09 (7 minutes and 9 seconds) to complete.

Request volume: total requests reached 696.

Stealth advantage: while the scan was much slower than previous attempts (which took seconds), the 0.5 second gap makes the traffic pattern appear more like a human browsing the site rather than an automated bot.

7.3 Security Rationale: Detection Avoidance

Implementing the --throttle flag is a direct measure to prevent the undesirable effects of being blocked by security infrastructure:

- Bypassing IDS/IPS: many intrusion detection systems look for a high frequency of 404 errors or rapid requests. Throttling keeps the request rate below typical alarm thresholds.
- Preventing rate limited: many web servers will automatically drop connections if they receive too many requests per second. Throttling ensures the scan can finish without being blacklisted by the target.
- Resource management: this approach is polite to the target server, ensuring that the security audit does not inadvertently cause a Denial of Service (DoS) by overwhelming the server's CPU or RAM/

Final Reflection

The combination of API-driven vulnerability mapping and throttled request delivery represents a more advanced and realistic approach to security assessment. This method not only improved the accuracy of identifying vulnerabilities through CVE mapping but also demonstrated how attackers can operate in a more discreet manner to avoid detection by security systems.

Throughout this lab, I developed practical skills in troubleshooting and tool configuration, particularly when resolving the 401 Unauthorized error during API integration. This experience highlighted the importance of understanding the underlying protocols and configurations, rather than relying entirely on automated tools. In addition, the identification of Critical Vulnerabilities such as SSRF, Cross-Site Scripting (XSS), and Directory Traversal reinforced how outdated components and insecure configurations can significantly increase the attack surface of a web application.

Real world attacks have demonstrate that the XML-RPC interface in WordPress can be exploited to perform brute-force amplification attacks, allowing multiple logins attempts within a single request and increasing the likelihood of bypassing detection mechanisms [12].

From a defensive perspective, the use of throttling illustrated how attackers can mimic normal user behaviour to bypass intrusion detection systems. This emphasised the need for more advanced monitoring techniques, such as behavioural analysis and anomaly detection, rather than relying solely on traditional rate-limiting controls.

Reference

[1] Editorial Staff, "What Are WordPress Plugins? And How Do They Work?," *WPBeginner*, 2023. [Online]. Available: https://www.wpbeginner.com/beginners-guide/what-are-wordpress-plugins-how-do-they-work/ [Accessed: Mar. 7, 2025].

[2] Patchstack, "State of WordPress Security 2025," *Patchstack*, 2025. [Online]. Available: https://patchstack.com/whitepaper/state-of-wordpress-security-in-2025/ [Accessed: Mar. 7, 2025].

[3] Trew Knowledge, "Why WordPress Isn't the Problem: A Real Look at Site Security," *Trew Knowledge Inc.*, 2023. [Online]. Available: https://trewknowledge.com/blog/wordpress-security/ [Accessed: Mar. 7, 2025].

[4] Kaspersky, "WordPress: Vulnerabilities in plugins and themes," *Kaspersky Official Blog*, 2024. [Online]. Available: https://www.kaspersky.com/blog/wordpress-plugin-vulnerabilities/ [Accessed: Mar. 7, 2025].

[5] Editorial Staff, "What is a WordPress Theme?," *WPBeginner*, Oct. 24, 2023. [Online]. Available: https://www.wpbeginner.com/glossary/theme/ [Accessed: Mar. 7, 2025].

[6] Tomas. A, "How to Find XSS (Cross-Site Scripting) Vulnerabilities in WordPress Plugins and Themes," *Wordfence Intelligence*, 2024. [Online]. Available: How To Find XSS (Cross-Site Scripting) Vulnerabilities in WordPress Plugins and Themes [Accessed: Mar. 7, 2026].

[7] CloudDefense.ai, "CVE-2022-3590: WordPress Core < 6.1.1 - Unauthenticated Blind SSRF," *CloudDefense Vulnerability Database*, 2024. [Online]. Available: https://www.clouddefense.ai/cve/2022/CVE-2022-3590 [Accessed: Mar. 7, 2026].

[8] National Vulnerability Database (NVD), "CVE-2023-2745 Detail," *NIST*, June 13, 2023. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2023-2745 [Accessed: Mar. 7, 2026].

[9] WP Security Ninja, "Understanding and Preventing WordPress XSS Attacks," *WP Security Ninja Blog*, Mar. 8, 2024. [Online]. Available: https://wpsecurityninja.com/understanding-and-preventing-wordpress-xss-attacks/ [Accessed: Mar. 7, 2026].

[10] National Cyber Security Centre (NCSC), "Denial of Service (DoS) Guidance Collection," *NCSC.gov.uk*, Feb. 20, 2024. [Online]. Available: https://www.ncsc.gov.uk/collection/denial-service-dos-guidance-collection [Accessed: Mar. 7, 2026].

[11] Legit Security, "Understanding Stored XSS: Risks and Prevention," *Legit Security ASPM Knowledge Base*, 2025. [Online]. Available: https://www.legitsecurity.com/aspm-knowledge-base/stored-xss [Accessed: Mar. 7, 2026].

[12] Sucuri, "Brute Force Amplification Attacks Against WordPress XML-RPC," Sucuri Blog, Oct. 2015. [Online]. Available: https://blog.sucuri.net/2015/10/brute-force-amplification-attacks-against-wordpress-xmlrpc.html. [Accessed: Mar. 10, 2026].